



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduzione al World Wide Web

Fabio Vitali

Corso di laurea in Informatica
Alma Mater – Università di Bologna

Introduzione

Oggi esaminiamo:

- Una breve storia del WWW
- I principali temi e tecnologie connesse con il web





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

II WWW

II WWW

- Gli inizi (1989-1994)
- La prima guerra dei browser (1995-99)
- Il predominio di Explorer (1999-2004)
- La seconda guerra dei browser (2004-2008)
- Il Ajax e il web 2.0 (2005-)
- L'ascesa dei dispositivi mobili (2009-)
- I social network (2004-)
- I framework a componenti (2013-)



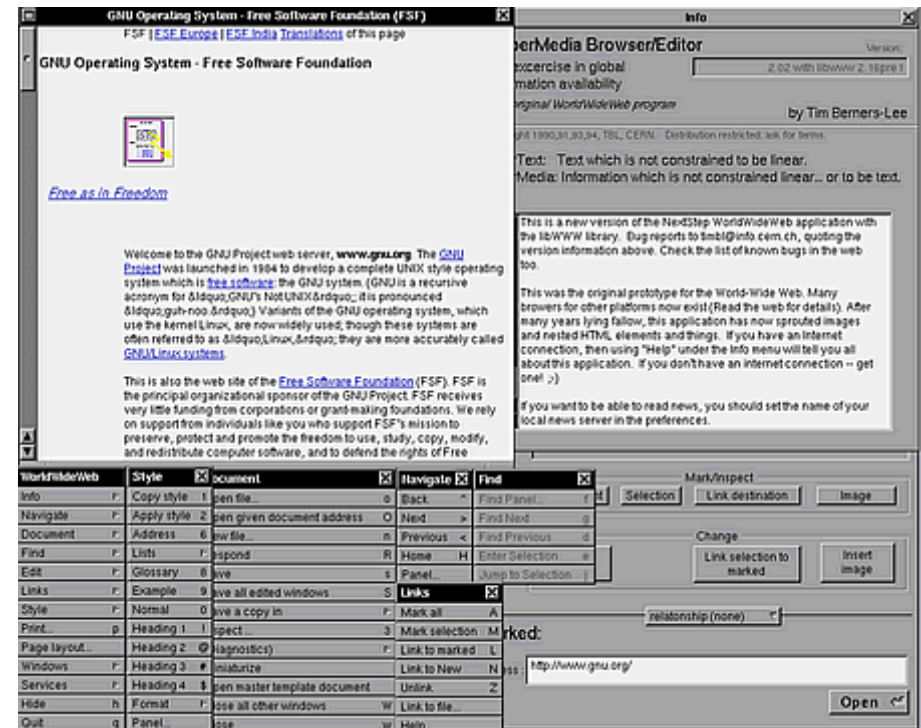
WWW: gli inizi (1)

Nel 1989, un gruppo di ricercatori informatici del CERN (il centro di ricerca in fisica nucleare di Ginevra) ricevettero l'incarico da parte della direzione di realizzare un meccanismo per la diffusione rapida di articoli, appunti e opinioni tra i fisici che ruotavano intorno al centro.

Tim Berners-Lee, Robert Cailliau ed altri identificarono Internet, ipertesti e SGML come elementi chiave per questo meccanismo.

Nel 1991, alla conferenza sugli ipertesti, Berners-Lee e Cailliau mostrarono (con poco successo) il primo prototipo della loro applicazione, realizzata in client-server su architettura NeXT (un precursore di Mac OS X): World-Wide Web.

Nonostante l'accoglienza fredda degli informatici, i fisici furono entusiasti del WWW: comodo, facile da usare e da imparare, gratuito, privo di alternative realistiche.



Il World Wide Web Consortium

Il W3C nasce dall'iniziativa di Tim Berners-Lee nel 1994 per promuovere e uniformare lo sviluppo delle tecnologie del Web. TBL ne è presidente a vita.

Il W3C dimostrò la sua importanza durante la lotta tra Netscape e Microsoft nel riuscire a generare un tavolo comune di discussione e integrazione delle feature introdotte in HTML dagli uni e dagli altri.

Successivamente, il W3C ha diretto lo sviluppo dei più importanti standard del Web: URI, HTTP, CSS. Un altro passo importante fu proporre e dirigere il gruppo di lavoro su XML e standard collegati, e quello sull'accessibilità sul web.

Durante la disputa sulla pornografia sul web, il W3C si schierò apertamente a favore della libertà di pubblicazione, identificò tecnologie alternative alla censura centralizzata (PICS, che poi evolverà nelle tecnologie del parental control) ed ebbe influenza determinante nel mantenere la libertà d'espressione sul web.



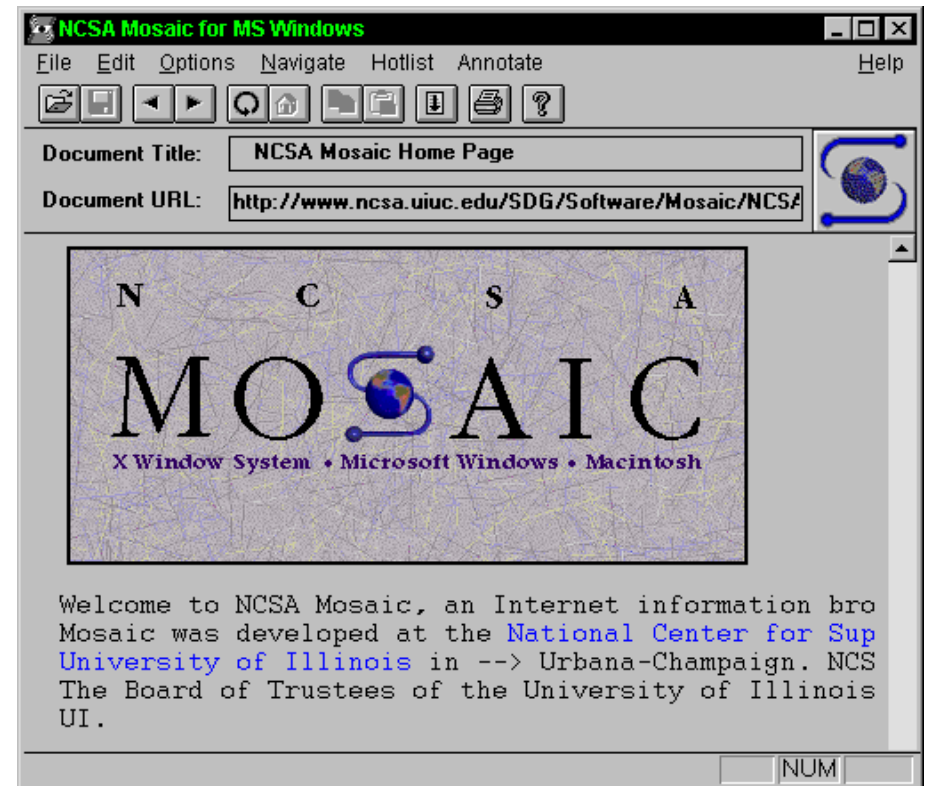
WWW: gli inizi (2)

Nell'ottobre del 1992 il National Centre for Supercomputing Applications (NCSA) decise di realizzare una versione propria di WWW. Il server NCSA ed il browser Mosaic furono l'inizio del successo su larga scala di WWW. Mosaic non aveva capacità di editing, ma aveva la capacità di visualizzare semplici immagini GIF.

In breve, milioni di utenti iniziarono ad usare Mosaic o altri browser WWW, e decine di migliaia installarono server HTTP. Nel frattempo, però, l'NCSA aveva deciso di non spendere più energie sul WWW.

Marc Andreessen, realizzatore del prototipo di Mosaic su Windows, si trovò la strada bloccata per l'evoluzione del progetto. Invece Jim Clark, ex professore a Berkeley e co-fondatore della Silicon Graphics, cercava una nuova idea da finanziare.

Andreessen e Clark fondano nel 1993 la Mosaic Corporation, poi rinominata Netscape per evitare problemi legali con NCSA. Il successo di Netscape Navigator (1994) è immediato e totale. La ditta Netscape ha il passaggio più rapido tra fondazione e quotazione in borsa della storia.



Storia del Web (3)

la prima guerra dei browser

Fin dall'inizio Clark e soci sanno di essere nel mirino di Microsoft, e si preoccupano di mantenere competitività e controllo del mercato.

Microsoft, dopo una falsa partenza con Microsoft Network, abbraccia definitivamente e con energia la tecnologia Internet, ed realizzare un browser (Internet Explorer) ed un server (Microsoft Information Server)



Sia Netscape che Microsoft introducono piccoli "miglioramenti" su HTML per migliorare l'appeal del proprio browser.

Per evitare la nascita di due internet separate e diverse, Berners-Lee e Cailliau fondano il W3C, con fondi della ricerca e dell'università per mantenere il controllo sull'evoluzione del World Wide Web. Berners-Lee viene fatto presidente *a vita*.



WWW: la prima guerra dei browser (4)

Netscape

- 1.0: ottobre 1994, clone esatto di Mosaic
- 2.0: settembre 1995
- 3.0: agosto 1996.

La concorrenza con Internet Explorer si fa sentire.

- 4.0: giugno 1997
- 4.08: Novembre 1998
- 4.7: settembre 1999

La meteora si conclude qui.

Internet Explorer

- 1.0: agosto 1995,
- 2.0: ottobre 1995. È preinstallata su tutti i computer venduti con Windows 95 Second Edition
- 3.0: 1996 con supporto per script e CSS.
- 4.0: ottobre 1997 è veramente veloce, compatibile e stabile.

Il vento cambia.



WWW: la dominanza di Explorer (5)

- Nel marzo 98 Netscape ammette la disfatta, rilasciando il codice sorgente della versione 5 di Navigator in open source e cercando di creare una comunità online.
- Nasce mozilla.org, e a novembre del 98 Netscape viene comprata da America On Line, Clark e Andreessen lasciano (multimiliardari).
- Dal 98 al 2001 non esce nessuna versione di Communicator e la versione di giugno del 2001 è lenta, pesante e piena di buchi.
- Da una costola di Communicator nasce Firefox, un browser nato per essere rapido e semplice e successivamente divenuto il prodotto principale di Mozilla.org

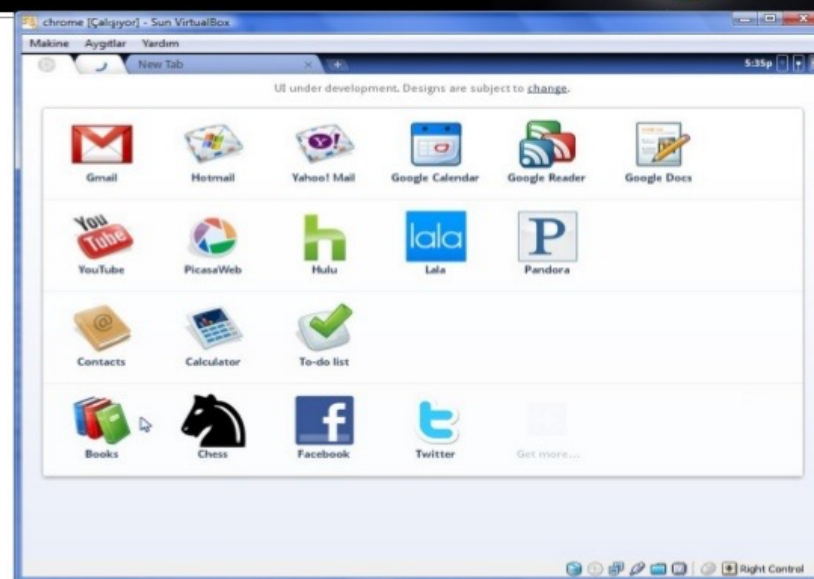


La seconda guerra dei browser

First Version (2009)



- Microsoft, dopo aver ottenuto il controllo del mercato dei browser, si disinteressa allo sviluppo di nuove funzionalità.
- In particolare, rifiutando di entrare nel gruppo WHAT, nel 2004, si è posta ad inseguire, più che a guidare, lo sviluppo delle tecnologie.
- Apple ha avuto una posizione molto più aggressiva, e insieme a Firefox e Opera, hanno riattivato il discorso sui browser.
- Alla fine del 2008 nasce Chrome di Google, stanca di dover dipendere dai produttori di browser per introdurre nuove funzionalità che gli servono.
- Diventa rapidamente il principale browser. Introduce anche il concetto di Rapid Release (piccoli incrementi ogni due-tre settimane), seguito da Firefox e Safari ma non da Microsoft.



Ajax e il web 2.0

- Internet Explorer rilascia un altro regalo: la libreria XMLHttpRequest (XHR) permette ad un'applicazione client-side di attivare richieste HTTP senza navigazione (cioé senza passare da una pagina HTML all'altra).
- Con l'affermazione di Gmail e Google Maps, tra 2004 e 2005, diventa evidente che le pagine possono aggiornarsi anche in parte senza navigare. Nel 2005 Jesse James Garrett battezza la tecnica **AJAX**.
- **XHR** è macchinosa e difficile da controllare. Si diffondono librerie come Prototype e poi jQuery, rendendo più semplice lo sviluppo di applicazioni Ajax.
- O'Reilly rilancia il termine **Web 2.0**: contenuti generati dagli utenti, API aperte, notifiche, mashup.
- XML (usato per default da XHR, e particolarmente verboso) viene sostituito da JSON e la logica applicativa si sposta sempre più sul client, e anche le interfacce si rendono più sofisticate, diventando sempre più simili a software nativi (desktop e mobile).



II WHAT-WG

- Nel 2004, Firefox e Opera proposero al W3C la riapertura del Working Group su HTML per lo sviluppo di nuove versioni del linguaggio. La proposta, ignorando volutamente XHTML e la rigida sintassi di XML, venne bocciata dal W3C.
- Venne allora formato un gruppo separato, chiuso e finanziato dalle società di software, il Web Hypertext Application Technology, che sviluppò proposte (Web Application 1.0) che vennero effettivamente implementate da vari browser. Queste modifiche riguardavano HTML, CSS, DOM e Javascript, e cambiavano radicalmente alcuni aspetti di stretta competenza del W3C.
- Nel 2007 il W3C dovette ammettere che queste modifiche avevano un impatto innegabile (“he who ships working code wins”) riaprì il working group con tutti i membri del WHAT per creare una nuova versione del linguaggio, HTML 5.



L'ascesa dei dispositivi mobili

- Nel 2007 Apple lancia l'iPhone che ha un browser integrato (Safari) e uno schermo molto più piccolo di un PC. Nasce il problema di fare siti web che funzionino in maniera adeguata su schermi di questo tipo.
- Nel 2008 escono i primi cellulari Android, creando esigenze di design "responsive" e "mobile first".
- Nascono e prosperano le app store e il traffico web migra sul mobile. Nel 2010 arriva l'iPad e seguono tablet Android.
- Nascono ecosistemi chiusi con policy e SDK proprietari (Apple/Google).



I social network

- **I social network:**
 - **2004–2012.** Facebook, YouTube, Twitter, Instagram, VKontakte.
 - **2013–2016:** WhatsApp, Snapchat, Telegram; acquisizione di Instagram; acquisizione WhatsApp.
 - **2016-oggi:** TikTok, Discord
- Spostano traffico e attenzione: il “Web aperto” perde centralità a favore di piattaforme chiuse e app native.
- Cambiano monetizzazione e UX: login social, feed algoritmici, tracciamento/ads, video-first e real-time.
- Alzano i requirement tecnica: CDN globali, streaming a bassa latenza, API/SDK proprietari; il web adotta tecniche di performance estreme per restare competitivo.



Da controllare gli utenti a controllare gli sviluppatori

- Le prime due guerre dei browser avevano come scopo quello di ottenere siti web che non funzionavano sui browser della concorrenza, spingendo gli utenti a scegliere il loro.
- Questo ha causato grosse obiezioni e, di fatto, la nascita del W3C come ambiente di mediazione e soluzione delle dispute. Inoltre ha dato una pessima fama al principale responsabile, Microsoft (Netscape non era innocente, ma ha perso).
- Google ed Apple hanno capito la lezione, e hanno progettato i loro browser come ambienti di esecuzione ***assolutamente generici***, tali per cui qualunque sito sufficientemente moderno può funzionare bene.
- Per vincere la guerra degli utenti, quindi, non si combatte più sulle feature scelte dall'utente e di cui egli si accorge, ma su quelle scelte dallo sviluppatore, che gli permettono di sviluppare più facilmente, più velocemente, più efficientemente siti web più sofisticati.
- In cambio lo sviluppatore accetta di bloccare in una specifica tecnologia il proprio sito (*technology lock-in*).



La guerra dei framework

- Ajax e il web 2.0 (2004-) facilitano la nascita delle Single Page Application (SPA) che però hanno tipicamente dei tempi di startup e latenza importanti.
- AngularJS (Google, 2012) e React (Facebook, 2013) sono framework quasi contemporanei che suggeriscono separatamente componenti dichiarativi nel markup, a cui associare codice e presentazione in maniera modulare.
- AngularJS esplode e si frammenta (2016): Angular 2 (senza .js) rinuncia a molti concetti originali e diventa una parte strategica della campagna "enterprise" di Google, mentre la parte più leggera e vicina alle origini (MVC, bidirectional binding, ecc.) viene adottato come Vue.js (2015) da una comunità agguerrita e molto attiva.
- Nel frattempo a toolchain JS esplode (typescript, node+ npm, typescript, browserify, webpack, moduli ES, cli, ide) e nascono ecosistemi interi fortemente integrati. Per ovviare alle lentezze di startup e latenza si torna al server: Next e Nuxt (2017) riportano Server-Side Rendering (SSR), seguiti da idratazione parziale (2020), streaming (2022) e edge rendering (2022).
- Ora iniziamo a ripensare a componenti predefiniti da integrare nelle pagine HTML, ridando il predominio al markup sul codice: HTMX (2020), Alpine (2019) e WebComponents (2016) riportano almeno in parte il focus sul markup.



Caratteristiche del lock-in tecnologico

"Technological lock-in is the idea that the more a group adopts a certain technology, the more unlikely they are to switch."

- Le tecnologie forniscono servizi utili e graditi agli utenti
- Le tecnologie hanno un costo ragionevole o sono gratuite
- Le tecnologie hanno una fortissima personalità, e cambiano radicalmente il modo di realizzare i prodotti che supportano
- Le tecnologie cambiano il prodotto stesso che supportano, in modo che non sia facile o possibile evolverlo, cambiarlo o aggiornarlo senza usare la tecnologia da cui dipendono.

L'utente finisce per diventare dipendente della tecnologia, non solo per l'abitudine e la piacevolezza del servizio offerto, ma anche e soprattutto per il costo (reale o percepito) connesso con il cambiamento ad altra tecnologia (*sunk cost*).



Framework di componenti

I framework dei componenti cambiano lo sviluppo di applicazioni Web tradizionali in molti modi:

- Un componente può essere associato a un elemento di markup inventato (i programmatori possono creare il proprio markup)
- I componenti isolano e raggruppano aspetti molto diversi dello sviluppo web: React ha inventato una nuova sintassi / linguaggio di programmazione di markup, chiamato JSX, che mescola tutto in un unico file.
- Angular ha adottato e sta spingendo molto una variante di Javascript, *Typescript*, che ha alcuni vantaggi di programmazione rispetto al puro JS, ma deve essere compilata prima dell'esecuzione.
- Le applicazioni di base dei componenti su vasta scala utilizzano centinaia di componenti più piccoli per funzionare, con centinaia di piccoli download dal server al client. Al fine di ridurre i tempi di caricamento, sono stati introdotti tool chiamati *packer*, che raggruppano e ottimizzano ciò che viene effettivamente utilizzato sul browser: problema: il codice che scrivi non è più il codice che viene eseguito sul browser (il debug è complesso).
- Le interfacce CLI (interfaccia a riga di comando) esistono per facilitare lo sviluppo, la manutenzione e la distribuzione di applicazioni complesse, ma non sono semplici da gestire.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Architettura del WWW

Cos'è il WWW

Il World Wide Web è un sistema "per la presentazione a schermo di documenti multimediali, e per l'utilizzo di link ipertestuali per la navigazione".

Il sistema è distribuito e scalato su tutta Internet, ed è basato su alcuni semplici concetti:

- Il *client* o browser è un visualizzatore di documenti ipertestuali e multimediali. Può visualizzare testi, immagini e semplici interfacce grafiche, ma non editare documenti (salvo trucchi).
- I browser hanno anche plug-in che permettono di visualizzare ogni tipo di formato speciale, e un linguaggio di programmazione interno (Javascript) che permette di realizzare semplici verifiche di dati oppure complicate applicazioni autosufficienti (Rich client)
- Il server è un meccanismo di accesso a risorse locali (file, record di database, ecc.) in grado di trasmettere via socket TCP documenti individuati da un identificatore univoco
- Il server può collegarsi ad applicazioni server-side ed agire da tramite tra il browser e l'applicazione cosicché il browser diventa l'interfaccia dell'applicazione.



I protocolli fondamentali del WWW

Alla base di WWW ci sono i seguenti protocolli:

- Uno standard per identificare in maniera generale risorse di rete e per poterle specificare all'interno di documenti ipertestuali (chiamato URI).
- Un protocollo di comunicazione state-less e client-server per l'accesso a risorse ipertestuali via rete (chiamato HTTP).
- Un linguaggio per la realizzazione di documenti ipertestuali (HTML e ora XHTML) basato su SGML (e ora XML) e incentrato sulla possibilità di realizzare connessioni ipertestuali in linea nella descrizione strutturale del documento.



Le reti a livelli

- Progettare una rete significa affrontare molti problemi: mezzi fisici diversi, instradamento, affidabilità e compatibilità.
- La soluzione è un'architettura a livelli, dove ogni livello svolge un compito preciso e si appoggia a quello inferiore.
- I livelli vanno dal trasferimento fisico dei segnali fino alle applicazioni come web o posta elettronica.
- Ogni livello segue protocolli standard aperti, che garantiscono compatibilità e permettono a nuovi dispositivi di integrarsi facilmente.



Le reti a livelli

	WiFi	Ethernet <i>(e.g. rete cablata universitaria)</i>	Fibra ottica <i>(e.g. FTTH)</i>	Rete cellulare <i>(e.g. 4G/5G)</i>	Rete satellitare <i>(e.g. ViaSat o StarLink)</i>
Applicazione <i>(il tipo di servizio)</i>	Es. HTTP, HTTPS, ecc.	Es. HTTP, HTTPS, ecc.	Es. HTTP, HTTPS, ecc.	Es. HTTP, HTTPS, ecc.	Es. HTTP, HTTPS, ecc.
Trasporto <i>(gestisce errori e l'integrità della comunicazione)</i>	TCP, UDP	TCP, UDP	TCP, UDP	TCP, UDP	TCP, UDP
Rete <i>(gestisce indirizzi e routing tra nodi remoti)</i>	IP	IP	IP	IP	IP
Collegamento <i>(definisce come trasferire pacchetti tra nodi direttamente collegati)</i>	IEEE 802.11 (WiFi)	IEEE 802.3 (Ethernet)	GPON (Gigabit Passive Optical Network)	LTE (4G), NR (5G)	Protocolli proprietari (es. MAC ViaSat, StarLink)
Fisico <i>(codifica i bit in segnali fisici concreti)</i>	Modulazione radio (OFDM)	Segnali elettrici su doppini intrecciati	Modulazione di luce laser	Modulazione radio (OFDMA)	Modulazione radio (Ka/Ku)
Mezzo fisico <i>(canale usato per il trasferimento)</i>	Onde radio a 2.4, 5, 6 GHz tra device e router WiFi	Cavi in rame tra device e armadio di rete (rack)	Impulsi luminosi attraverso fibre di vetro flessibili	Onde radio tra device e antenna cellulare di zona (700 MHz–3.5+ GHz)	Onde radio (20–30 GHz) tra device e satellite geostazionari (ViaSat) o in orbita bassa (StarLink)

TCP-IP

- I livelli alti sono molto standardizzati: IP (Internet protocol) e TCP (TeleCommunication Protocol) oppure in casi particolare UDP (User Datagram Protocol).
- IP si occupa dell'instradamento e del delivery: sintassi dei nodi mittente, destinatari ed intermedi, e algoritmi di instradamento. Ogni datagramma può seguire percorsi diversi: possono verificarsi ritardi, riordinamenti o perdite.
- TCP fornisce affidabilità sopra IP: crea connessioni affidabili, gestisce la numerazione e ritrasmissioni dei pacchetti persi, ne ricompone l'ordine e gestisce le porte a cui far arrivare i pacchetti.
- UDP rinuncia alle garanzie per ridurre latenza e overhead: utile per VoIP, video live e gaming.
- Il DNS trasforma i nomi di dominio (facili da memorizzare) in indirizzi IP (richiesti dalla rete).



Evoluzioni del WWW (1)

- **Multimedia:** Mosaic introdusse il supporto per immagini in-line, e poi successivamente attraverso plug-in e viewer specializzati: video, suoni, PDF, ecc.
- **Client Scripting:** Netscape introdusse LiveScript, poi ribattezzato Javascript, per realizzare applicazioni client-side. IE rispose con Jscript e Vbscript. Sebbene standardizzato (da ECMA) per quel che riguarda sintassi e classi fondamentali, è sempre stato implementato diversamente da browser a browser.
- **Stili tipografici:** L'uso di trucchi per forzare HTML a rese grafiche insolite ha portato a creare linguaggi appositi per gestire gli aspetti di visualizzazione del documento. CSS permette di controllare le caratteristiche dei documenti HTML.



Evoluzioni del WWW (2)

- ***Gestione delle transazioni***: meccanismi per la gestione dello stato sono stati introdotti prima da Netscape, e poi standardizzati (cookies). Anche XMLHttpRequest diventa universale.
- ***Applicazioni web 1.0***: le applicazioni server-side (CGI-BIN) evolvono e si fondono, diventano veri e propri linguaggi o ambienti di programmazione: Perl prima, poi ASP (Javascript e Visual Basic), poi PHP, Python, Ruby, ecc.
Nasce l'architettura a tre livelli (user-interface, application logic, data storage). Il browser diventa l'interfaccia di eccellenza per applicazioni gestionali distribuite.



Evoluzioni del WWW (2)

- ***Strutturazione dei documenti***: i limiti di HTML non erano soltanto nella visualizzazione, ma anche nella strutturazione. XML ha permesso di definire linguaggi di markup più adatti ai singoli task.
- ***Framework di sviluppo***: ambienti integrati di sviluppo dotati di ricche API e librerie per la realizzazione semplificata di applicazioni client-side e server-side. Sono particolarmente noti Django per Python, Rails per Ruby, tutti i framework Ajax (Prototype, JQuery, Ext, Dojo, ecc.). Interessante anche l'esperienza di Google Web Toolkit (GWT), un framework Ajax /Javascript espresso in Java, che integra modelli di progettazione delle parti client-side e server-side.



Mode del passato e del presente (1)

Trucchi di HTML

- Il linguaggio HTML non nasce per ottenere speciali effetti grafici. In pratica, HTML gestisce poco le caratteristiche di skin e per niente quelle di layout.
- Tuttavia i progettisti di siti web trovavano queste limitazioni frustranti e inaccettabili. Dunque inventarono svariati trucchi, che tuttora si vedono, per ottenere certi effetti grafici sulle pagine web:
 - Immagini di testo invece che testo (soprattutto nei bottoni)
 - Tabelle di layout
 - Spacer (single-pixel gif)
 - Blockquote
 - Il tag
 - I frame
 - Estensioni di singoli browser (<marquee>)
- Tutte queste tecniche sono ormai obsolete e ingiustificate, e debbono essere sostituite con le corrispondenti tecniche standard.



Mode del passato e del presente (2)

LAMP

- La risposta open source al modello DHTML/ASP di Microsoft:
 - Linux
 - Apache
 - MySQL
 - Perl (successivamente anche PHP e adesso anche Python)
- Uno stack di linguaggi e protocolli per la realizzazione di siti web dinamici e interattivi, con memorizzazione delle informazioni su un database relazionale, separazione di memorizzazione, logica e distribuzione.

REST

- Representational State Transfer: il modello di interazione tra client e server proposto da HTTP e spesso tradito in varie implementazioni. Una corretta applicazione di REST garantisce interoperabilità, scalabilità, e uso delle caratteristiche di HTTP più avanzate.



Mode del passato e del presente (3)

Semantic Web

- la attribuzione di semantica ai dati per la creazione e la condivisione di un modello concettuale delle informazioni. Molto ambizioso, forse condannato proprio dall'ambizione.

Linked Data

- Un modello molto più modesto di attribuzione di semantica e indirizzabilità ai dati su cui sia possibile costruire applicazioni sofisticate e fortemente connesse anche in assenza di modelli concettuali forti.

Open Linked Data

- La versione politicizzata e politicamente corretta del Linked Data su dati resi disponibili dalla Pubblica Amministrazione con modalità Linked Data.



Mode del passato e del presente (4)

Ajax

- Asynchronous Javascript And XML: un meccanismo per generare applicazioni web client-side e server-side fortemente interattive e in grado di minimizzare il traffico di rete.

Node.js

- Un eseguibile basato sul motore di Chrome (V8) che permette di eseguire codice Javascript anche server-side. Ha rivoluzionato la progettazione di applicazioni web server-side. Insieme a *npm*, il loader di moduli per Node, ha creato una varietà ricchissima di estensioni e servizi per applicazioni web mai viste prima.

MEAN/MERN stack

- I nuovi stack di linguaggi e tecnologie per il web, di gran moda adesso:
 - MongoDB
 - ExpressJS
 - Angular
 - NodeJs



Mode del passato e del presente (5)

Mobile first

- lo spostamento dell'attenzione verso il supporto primariamente per le piattaforme mobili (smartphone e tablet) e solo secondariamente per i PC tradizionali.

Responsive web design

- La grande varietà di device, schermi, periferiche ecc. fa sì che una pagina con una geometria rigida e progettata per una specifica dimensione funzioni male per la maggior parte degli utenti. Una pagina responsive si adatta elegantemente a qualunque tipo di schermo al meglio delle sue possibilità.

Single Page Applications

- La varietà di contenuti e servizi di un sito di medie dimensioni non è più organizzato in pagine grandi come la finestra e raggiungibili tramite link, ma in un'unica pagina molto lunga e scrollabile. Un'unico indirizzo in cui convivono dati e servizi.



Mode del passato e del presente (7)

Component-based framework

- Una pagina web, specialmente una pagina complessa e con molti elementi diversi (interfaccia, contenuto, layout, servizi, ecc.) richiede la coordinazione di parti visuali, markup, stile, computazione, ecc.
- Nel component-based design queste parti vengono sviluppate in maniera integrata per ogni componente, e segregata tra componente e componente.
- Quindi un componente contiene frammenti di HTML (spesso chiamati *template*), classi CSS e codice Javascript, complessivamente autosufficienti e necessari per il buon funzionamento del componente.
- Introdotto dal framework AngularJS, è diventato estremamente popolare negli ultimi anni, creando altri framework estremamente popolari e famosi, tra cui Angular (derivato da AngularJS e supportato da Google), React (supportato da Facebook) e VueJs (gestito da alcuni autori di AngularJS che si sono messi in proprio).



Mode del passato e del presente (7)

Typescript, CLI, WebPack, browserify

- Con il tempo le funzionalità spartane delle varie tecnologie di base risultano inadeguate. L'approccio *document-centric* del web tradizionale non è capito dai programmatori che vengono da altre tradizioni. L'esigenza di applicazioni efficienti e offuscate prevale sulla trasparenza ed apertura precedenti. Si diffondono nuove tecnologie:
 - Typescript è un linguaggio che transpila in Javascript e aggiunge controlli sui tipi e altre cose. Molto apprezzato da Google, obbligatorio per Angular.
 - Command Line Interface sono strumenti da command line (e quindi cmd.exe o shell Linux), scriptabili e componibili, che rendono la configurazione di un progetto web particolarmente veloce e aggiornabile automaticamente
 - Webpack è un web bundler, ovvero un compilatore di applicazioni in tecnologie varie che genera un'applicazione web eseguibile. Permette di compilare, riorganizzare il codice, introdurre automaticamente librerie e dipendenze, offuscare il codice.
 - Browserify è uno strumento che permette di importare nel browser librerie pensate per Node e npm. Ha spostato il focus della progettazione dal client al server.



Mode del passato e del presente (6)

Progressive web applications

- Termine usato da Google per applicazioni web che vengono sviluppate e caricate come normali pagine web, ma che si comportano come applicazioni native quando utilizzate su un dispositivo mobile.

Opinionated vs. non-opinionated frameworks

- I framework sono librerie che semplificano radicalmente la progettazione di pagine e servizi web, e uniformano le differenze dei singoli browser. Tuttavia i framework possono essere più o meno rigidi e restrittivi nell'imporre un modello di design specifico.

Static site generators

- L'eccesso di granularità nei siti web basati su componenti ha fortemente ridotto la velocità di caricamento delle applicazioni web più complesse: centinaia di file diversi, esecuzione di migliaia di righe di codice prima di mostrare una cosa qualunque all'utente, ecc.
- Gli static site generator sono applicazioni che pre-eseguono questi script server-side e generano un sito web "statico" come una volta, solo che come risultato di script moderni.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Corsi e ricorsi storici

Corsi e ricorsi storici

- Markup chiuso o aperto
- Markup presentazionale o semantico
- Edit-Save-Reload vs. build and deploy
- Il valzer delle dipendenze
- La rumba del riuso
- Il tango delle integrazioni forzate



Markup chiuso o aperto

- SGML e XML sono *meta-linguaggi* di markup: attraverso di loro si definiscono linguaggi di markup
 - HTML: linguaggio ipertestuale per documenti di rete.
 - Text Encoding Initiative: documenti letterari o storici di varia natura.
 - DocBook: standard editoriale delle case editrici internazionali.
 - JATS: evoluzione di docbook specifico per riviste mediche.
 - Akoma Ntoso: documenti legislativi prodotti o pubblicati da parlamenti e governi nazionali e locali.
- HTML si è nel tempo arrogato il compito di gestire TUTTE le esigenze di rappresentazione di contenuti testuali, un linguaggio di markup universale utile per tutti i tipi di documento.
- Con la nascita dei componenti, adesso è possibile personalizzare HTML con le proprie estensioni e creare nuovamente set di elementi specifici per classi.



Markup presentazionale o semantico

- SGML introduce l'idea del markup semantico: io individuo nel documento gli elementi strutturalmente o semanticamente significativi:
 - Ad esempio: in una ricetta di cucina: gli ingredienti, i tempi, le quantità, i passi di preparazione, ecc.
- HTML descrive inizialmente un markup per lo più presentazionale:
 - paragrafi, i grassetto, i corsivi, le tabelle, le righe, le celle, ecc.
- XML ripropone l'idea che il metalivello sintattico è semantico e non presentazionale e suggerisce la separazione tra XHTML e CSS.
- WhatWG ritiene predominante la struttura, più che la semantica, e assegna al CSS la presentazione.
- I framework a componenti reintroducono l'idea di nuovo markup associato a esigenze strutturali o semantiche, reintroducendo l'idea di creare linguaggi (=collezioni di componenti) omogenei per classi di documenti.



Edit-save-reload vs. build and deploy

- La programmazione professionale, da sempre, prevede due ambienti, staging e deploy, e un processo di conversione del codice sorgente in codice utilizzabile (compilazione, transpilazione, pacchettizzazione, copia, ecc.)
- La prima versione del web prevedeva invece il processo molto più semplice: apro il documento HTML. lo modifico, salvo il risultato, vado sul browser, ricarico il documento e guardo se mi soddisfa (Edit-Save-Reload).
- Con l'avvento delle librerie Javascript sempre più complesse, l'ambiente di staging ritorna in auge, e il deploy sempre più complesso
 - Posso fare una pagina HTML con poche decine di byte
 - L'applicazione minima Angular parte da 300Kb, quella di React da 180Kb

L'uso di un processo automatico di build e deploy diventa indispensabile.

- HTMX e Alpine ritornano a dare predominanza al documento HTML: script anche complessi possono essere ridotti a estensioni di markup che non richiedono grandi librerie e processi di build: ritorna ad avere senso l'Edit-Save-Reload.



Il valzer delle dipendenze

- Nel processo di produzione moderno, con Angular o React o Vue, l'utilizzo di librerie esterne da inserire nel proprio codice è diventato universale.
- NPM, il gestore di pacchetti di Node.js, si è trasformato con successo in un ecosistema di librerie con dipendenze multiple.
 - **Angular (CLI nuovo progetto)**: circa 1.000–1.200 pacchetti npm installati, derivati da circa 60–70 dipendenze dirette: Angular core (framework, router, forms, RxJS, Zone.js), TypeScript (tslib, types), build (Angular CLI, builder, Vite/esbuild, terser, source-maps), CSS (PostCSS, Autoprefixer), test (Jest/Playwright o equivalenti), lint/tooling (ESLint, schematics). Ogni funzione spesso è un pacchetto separato: polyfill, formati, parser, glob, color, logging, ecc.
 - **React (creato con Vite o Create React App)**: 300–500 pacchetti totali, con 10–20 dipendenze dirette.
- Il caso *leftpad* (2026): una libreria banalissima di 11 righe (aggiunge spazi a sinistra di un numero fino ad arrivare ad un numero fisso di caratteri) viene rimossa per un litigio tra lo sviluppatore e GitHub. Centinaia di librerie (tra cui Babel e React, non possono più fare build e migliaia di progetti si bloccano senza spiegazioni per colpa di una dipendenza di una dipendenza di una dipendenza.



La rumba del riuso

- Una azienda di produzione software a commesse ha un grande interesse a crearsi una propria libreria di elementi e moduli riutilizzabili in più progetti.
 - Diminuisce lo sforzo di produzione di nuovi moduli e quindi i costi associati.
 - Diminuisce lo sforzo di aggiornamento dei moduli in seguito a evoluzioni del software o delle architetture di riferimento, e quindi i costi associati
 - Aumenta la dipendenza dei clienti dal proprio software (solo loro sanno mettere le mani nelle librerie e negli elementi che hanno prodotto)
- Tuttavia micro-variazioni tra cliente e cliente possono generare vaste collezioni di classi "quasi" uguali da mantenere e evolvere.
- La pigrizia e l'ignoranza faranno creare collezioni molto simili ma leggermente diverse interne alla stessa azienda impendendo il riuso e di approfittare delle economie di scala desiderate.
- L'uso di schemi e modelli concettuali che a priori identificano i componenti e quindi le classi appropriate, magari sufficientemente generiche da essere usabili in progetti diversi, potrebbe risolvere molti problemi. L'analisi strutturale e semantica di classi e componenti torna ad essere appropriata: meta linguaggi di componenti.



Il tango delle integrazioni forzate

- Decine di volte nella storia recente sono uscite librerie che proponevano funzionalità generiche di grande utilità e di grande successo.
- Con calma i browser chiedevano ed ottenevano che queste funzionalità (magari con nomi e sintassi diversa) venissero integrate nel linguaggio "ufficiale".
 - JQuery (2006) ha proposto selettori DOM arbitrari usando la sintassi CSS. JS ha creato querySelectorAll nel 2010.
 - JQuery (2006) ha proposto un meccanismo semplificato e asincrono per fare richieste Ajax. Axios fa servizi simili nel 2014. JS ha creato fetch() nel 2016.
 - Promise (2012) ha creato il concetto stesso di promessa (programmazione asincrona s comprensibile e facile da implementare), adottata come Promise nel 2015 e come async/await nel 2017
 - RequireJS introduce la modularità JS nel 2009, i moduli JS diventano parte del linguaggio nel 2015 (server) e 2017 (client)
 - Lodash (2012) decine di funzioni di varia utilità (Object.assign, Array.from, Object.entries, ecc.) adottati tra il 2015 e il 2019.
- L'adozione significa un grande onore ma anche lo svuotamento dell'utilità della libreria.



Conclusioni

- Qui abbiamo parlato di
 - I principali temi e tecnologie connesse con il web





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Corso di tecnologie web, A.A. 2017-18

Fabio.vitali@unibo.it

www.unibo.it